

METHOD 3

Jüttner, D., Labarthe, S., and Prastalo, N.

2026

1. CODEBOOK

- `iso3c` - ISO 3-letter country code
- `year` - Tournament year
- `v2exl_legitideolcr_0_1` - Nationalism (1 year lead)
- `v2exl_legitideolcr_0_3` - Nationalism (3 year lead)
- `treatment` - World Cup qualification treatment
- `points_gap` - Points gap between last qualifier and first non-qualifier
- `points_diff` - Assignment variable (points margin from cutoff)
- `wdi_expmil` - Military expenditure (% of GDP)
- `wdi_pop_log` - Total population (log)
- `wdi_popurb` - Urban population
- `wdi_lifexp` - Life expectancy at birth
- `wdi_export` - Exports of goods and services (% of GDP)
- `wdi_import` - Imports of goods and services (% of GDP)
- `wdi_gdpcapcon2015` - GDP per capita (constant 2015 US\$)
- `wdi_gdpgr` - GDP growth (annual %)
- `wdi_area` - Land area (sq. km)
- `wpp_medianage` - Median age of population
- `wpp_sexratio` - Sex ratio (males per 100 females)
- `vdem_polyarchy` - Polyarchy (electoral democracy index)

2. DATABASE REPLICATION

```
## a. VDEM
library(dplyr)
library(countrycode)
library(countries)
library(tidyr)
library(readr)

vdem <- read_rds("https://raw.githubusercontent.com/SimonLabarthe/Method-3/main/vdem.rds") %>%
  mutate(v2exl_legitideolcr_0_1 = lead(v2exl_legitideolcr_0, 1),
         v2exl_legitideolcr_0_3 = lead(v2exl_legitideolcr_0, 3))

vdem_clean <- vdem %>%
  mutate(iso3c = country_name(vdem$country_name,
                             to = "ISO3",
                             fuzzy_match = TRUE,
                             verbose = FALSE)) %>%
  filter(!is.na(iso3c))
```

```

## b. QOG
url_data <- "https://raw.githubusercontent.com/SimonLabarthe/Method-3/main/qog.rds"
qog <- read_rds(url_data) %>%
  rename(country_name = cname) %>%
  select(
    country_name,
    year,

    wdi_expmil,          # Military expenditure % of GDP (i = 163, t=1960-2024))

    wdi_pop,            # Total Population (i = 192, t = 1960-2024)
    wdi_popurb,         # Urban Population (i = 192, t = 1960-2024)
    wdi_lifexp,         # Life Expectancy at birth (per 1,000 live births) (i = 192, t = 1960-2023)

    wpp_medianage,     # Median Age as of 1 July (WPP) (i = 192, t = 1950-2024)
    wpp_sexratio,      # Sex Ratio: males per 100 females (WPP) (i = 192, t = 1950-2024)

    wdi_export,        # Exports of goods & services (% of GDP) (i = 173, t = 1960-2024)
    wdi_import,        # Imports of goods & services (% of GDP) (i = 173, t = 1960-2024)
    wdi_gdpcapcon2015, # GDP per capita (constant 2015 US dollar) (i = 191, t = 1960-2024)
    wdi_gdpgr,         # GDP growth (annual %) (i = 191, t = 1961-2024)

    wdi_area,          # Land area in sq. km (i = 191, t = 1961-2023)

    vdem_polyarchy     # Democracy level (i = 175, t = 1946-2024)
  ) %>%
  mutate(
    wdi_pop_log = log(wdi_pop)
  )

qog_clean <- qog %>%
  mutate(iso3c = country_name(country_name, # here again
                                to = "ISO3",
                                fuzzy_match = TRUE,
                                verbose = FALSE)) %>%
  filter(!is.na(iso3c))

## c. FIFA World Cup
library(tidyverse)
library(worldcup)
library(lubridate)

data("matches")
data("tournaments")
data("teams")
data("qualified_teams")

qualification_matches <- matches %>%
  filter(stage_name == "group stage") %>%
  mutate(
    year = as.numeric(str_extract(tournament_name, "\\d{4}"))
  ) %>%
  filter(year %% 2 == 0) %>% # males only

```

```

mutate(
  home_team_name = if_else(home_team_name == "England", "United Kingdom", home_team_name), # England
  home_iso3c = country_name(home_team_name, to = "ISO3", fuzzy_match = TRUE, verbose = FALSE),

  away_team_name = if_else(away_team_name == "England", "United Kingdom", away_team_name),
  away_iso3c = country_name(away_team_name, to = "ISO3", fuzzy_match = TRUE, verbose = FALSE)
) %>%
filter(!is.na(home_iso3c), !is.na(away_iso3c))

home_team_matches <- qualification_matches %>%
select(
  tournament_id,
  iso3c = home_iso3c,
  year, match_id, match_date,
  group_name, group_stage, stage_name,
  country_name = home_team_name,
  opponent_name = away_team_name,
  goals_for = home_team_score,
  goals_against = away_team_score,
  win = home_team_win, draw, loss = away_team_win
) %>%
mutate(points = case_when(win == 1 ~ 3, draw == 1 ~ 1, TRUE ~ 0),
  goal_difference = goals_for - goals_against,
  venue = "home")

away_team_matches <- qualification_matches %>%
select(
  tournament_id,
  iso3c = away_iso3c,
  year, match_id, match_date,
  group_name, group_stage, stage_name,
  country_name = away_team_name,
  opponent_name = home_team_name,
  goals_for = away_team_score,
  goals_against = home_team_score,
  win = away_team_win, draw, loss = home_team_win
) %>%
mutate(points = case_when(win == 1 ~ 3, draw == 1 ~ 1, TRUE ~ 0),
  goal_difference = goals_for - goals_against,
  venue = "away")

monadic_matches <- bind_rows(home_team_matches, away_team_matches)

qualification_standings <- monadic_matches %>%
group_by(year, group_name, iso3c) %>%
summarise(
  matches_played = n(),
  wins = sum(win, na.rm = TRUE),
  draws = sum(draw, na.rm = TRUE),
  losses = sum(loss, na.rm = TRUE),
  goals_for = sum(goals_for, na.rm = TRUE),
  goals_against = sum(goals_against, na.rm = TRUE),
  goal_difference = sum(goal_difference, na.rm = TRUE),

```

```

    points = sum(points, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(year, group_name, desc(points), desc(goal_difference), desc(goals_for))

qualification_standings <- qualification_standings %>%
  group_by(year, group_name) %>%
  mutate(
    rank = row_number(),
    teams_qualifying = if_else(year <= 1970, 1, 2), # number of qualified teams
    qualified = if_else(rank <= teams_qualifying, 1, 0)
  ) %>%
  ungroup()

# filter : keep only tight races
tight_races <- qualification_standings %>%
  group_by(year, group_name) %>%
  mutate(
    min_qualifier_points = min(points[qualified == 1], na.rm = TRUE),
    max_non_qualifier_points = max(points[qualified == 0], na.rm = TRUE),
    points_gap = min_qualifier_points - max_non_qualifier_points
  ) %>%
  ungroup() %>%
  filter((qualified == 1 & points == min_qualifier_points) |
    (qualified == 0 & points == max_non_qualifier_points)) %>%
  mutate(
    treatment = qualified,
    points_diff = if_else(qualified == 1, points_gap, -points_gap)
  )

## d. Merge
data <- tight_races %>%
  left_join(qog_clean, by = c("iso3c", "year")) %>%
  left_join(vdem_clean, by = c("iso3c", "year")) %>%
  filter(!is.na(treatment),
    points_diff != "Inf")

data <- data %>%
  select(iso3c, year, v2exl_legitideolcr_0_1, v2exl_legitideolcr_0_3, treatment, points_gap, points_diff)

library(labelled)

var_labels <- c(
  "iso3c" = "ISO 3-letter country code",
  "year" = "Tournament year",
  "v2exl_legitideolcr_0_1" = "Nationalism (1 year lead)",
  "v2exl_legitideolcr_0_3" = "Nationalism (3 year lead)",
  "treatment" = "Treatment: World Cup qualification (1=qualified, 0=non-qualified)",
  "points_gap" = "Points gap between last qualifier and first non-qualifier",
  "points_diff" = "Assignment variable: points margin from cutoff (positive=qualified)",
  "wdi_expmil" = "Military expenditure (% of GDP)",
  "wdi_pop_log" = "Total population (log)",
  "wdi_popurb" = "Urban population",

```

```

"wdi_lifexp" = "Life expectancy at birth",
"wdi_export" = "Exports of goods and services (% of GDP)",
"wdi_import" = "Imports of goods and services (% of GDP)",
"wdi_gdpcapcon2015" = "GDP per capita (constant 2015 US$)",
"wdi_gdpgr" = "GDP growth (annual %)",
"wdi_area" = "Land area (sq. km)",
"wpp_medianage" = "Median age of population",
"wpp_sexratio" = "Sex ratio (males per 100 females)",
"vdem_polyarchy" = "Polyarchy (electoral democracy index)"
)

data <- labelled::set_variable_labels(data, .labels = var_labels)

look_for(data)

test <- data %>%
  group_by(iso3c, year) %>%
  slice(1) %>%
  na.omit()

saveRDS(data, "data.rds")

```

3. LOAD DATA IN CLEAN ENVIRONMENT

```

library(dplyr)

## Warning: le package 'dplyr' a été compilé avec la version R 4.5.3
##
## Attachement du package : 'dplyr'
## Les objets suivants sont masqués depuis 'package:stats':
##
##     filter, lag
## Les objets suivants sont masqués depuis 'package:base':
##
##     intersect, setdiff, setequal, union

library(readr)
library(ggplot2)

## Warning: le package 'ggplot2' a été compilé avec la version R 4.5.3

rm(list=ls())

url_data <- "https://raw.githubusercontent.com/SimonLabarthe/Method-3/main/data.rds"

data <- read_rds(url_data) %>%
  mutate(y1 = v2exl_legitideolcr_0_1,
         y3 = v2exl_legitideolcr_0_3,
         d = treatment,
         z = points_diff) %>%
  select(iso3c,
         year,

```

```

    y1,
    y3,
    d,
    z,
    everything(),
    -v2exl_legitideolcr_0_1,
    -v2exl_legitideolcr_0_3,
    -treatment,
    -points_diff)

x <- data[, 8:19]

```

4. ASSUMPTIONS CHECK

a. Covariates Visualization P-Value

```

library(ggplot2)
library(gridExtra)

##
## Attachement du package : 'gridExtra'
## L'objet suivant est masqué depuis 'package:dplyr':
##
##      combine

data_c0 <- data %>% filter(data$z == 0) # on the cutoff
data_bw1 <- data %>% filter(data$z >= -1 & data$z <= 1) # around the cutoff

# datasets to test
datasets <- c("data", "data_bw1", "data_c0") # multiple datasets possible

# Covariates
vars <- names(x)

# add labs
better_names <- sapply(vars, function(var) {
  label <- attr(data[[var]], "label")
  if(is.null(label)) var else label
})

plot_list <- list()

# plot the p-values
for(d in datasets) {
  data_current <- get(d)
  balance_p <- rep(NA, length(vars))

  for(i in seq_along(vars)){
    temp_mod <- lm(
      as.formula(paste(vars[i], "~ d")),
      data = data_current[, c(vars[i], "d")]
    )
    balance_p[i] <- summary(temp_mod)$coefficients[2, 4]
  }
}

```

```

}

# new dataframe for graph
df <- data.frame(vars = vars, balance_p = balance_p, better_names = better_names)
df$y <- rank(-df$balance_p, ties.method = "first")

# extract p-values
cat("\n===== Dataset:", d, "=====\n")
print(data.frame(Variable = better_names, P_value = round(balance_p, 4)))

# graph
p <- ggplot(df, aes(x = balance_p, y = y)) +
  geom_point(size = 2, color = "darkblue") +
  scale_x_continuous(expand = c(0,0), limit = c(-0.01, 1.01),
    breaks = c(0.05, 0.1, 0.25, 0.5, 0.75, 1)) +
  xlab(expression(paste(italic("p"), "-value", sep = ""))) +
  scale_y_continuous(breaks = df$y, labels = df$better_names) +
  ylab(NULL) +
  geom_vline(xintercept = 0.05, lty = 2, color = "red", linewidth = 0.8) +
  geom_vline(xintercept = 0.10, lty = 2, color = "orange", linewidth = 0.8) +
  theme_bw() +
  theme(panel.grid.major.x = element_blank(),
    panel.grid.minor = element_blank(),
    panel.grid.major.y = element_line(linewidth = 0.15, linetype = "dashed"))

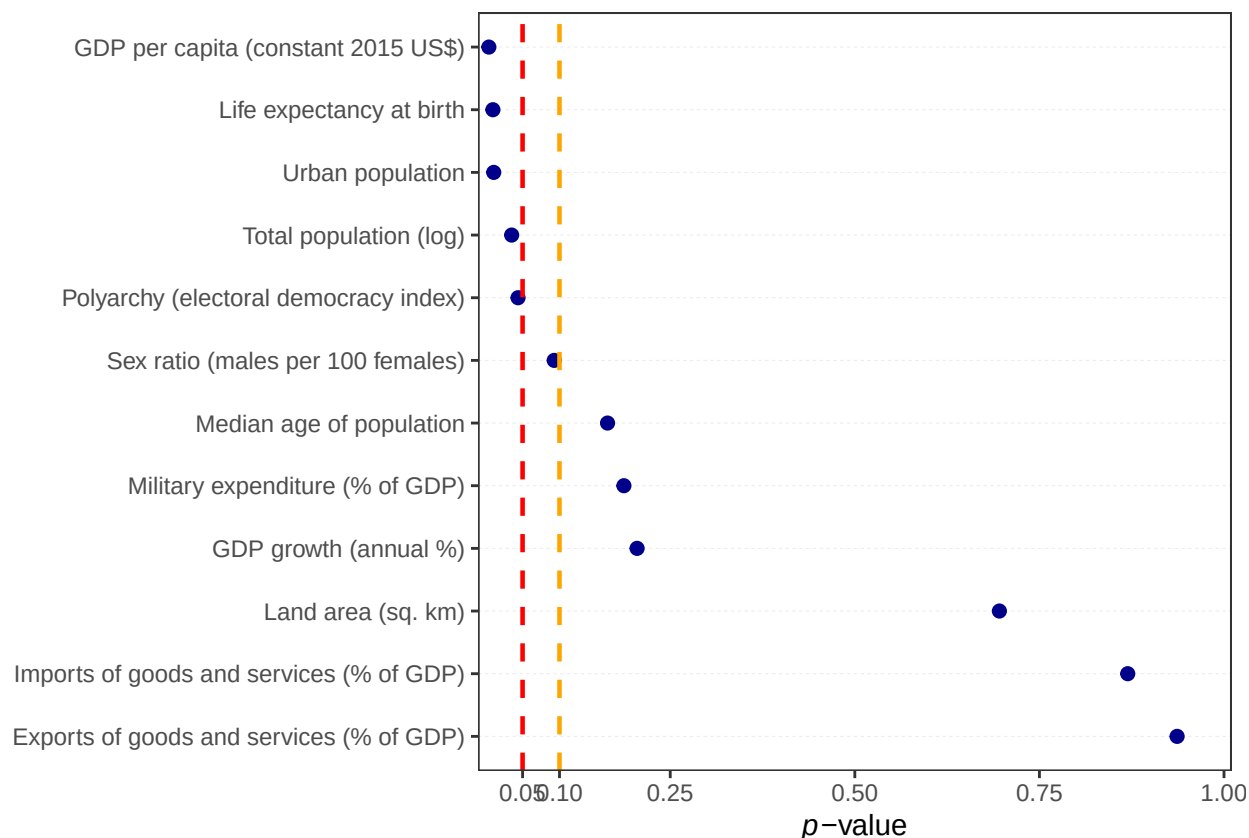
print(p)
plot_list[[d]] <- p
ggsave(paste0("balance_plot_", d, ".png"), p, width = 10, height = 6)
}

```

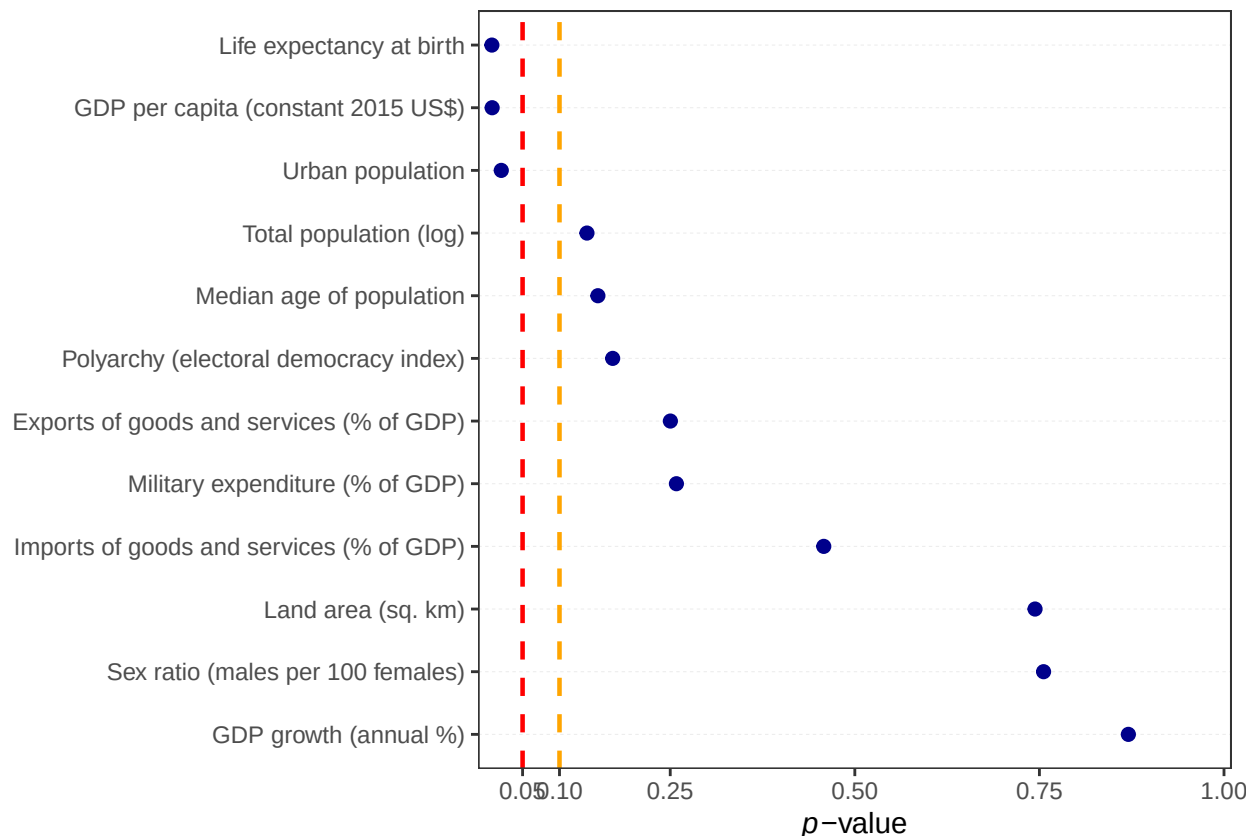
```

##
## ===== Dataset: data =====
##
##                                     Variable P_value
## wdi_expmil           Military expenditure (% of GDP) 0.1872
## wdi_pop_log           Total population (log) 0.0352
## wdi_popurb            Urban population 0.0109
## wdi_lifexp            Life expectancy at birth 0.0098
## wdi_export            Exports of goods and services (% of GDP) 0.9365
## wdi_import            Imports of goods and services (% of GDP) 0.8696
## wdi_gdpcapcon2015     GDP per capita (constant 2015 US$) 0.0043
## wdi_gdpgr            GDP growth (annual %) 0.2051
## wdi_area             Land area (sq. km) 0.6958
## wpp_medianage        Median age of population 0.1651
## wpp_sexratio         Sex ratio (males per 100 females) 0.0928
## vdem_polyarchy       Polyarchy (electoral democracy index) 0.0439

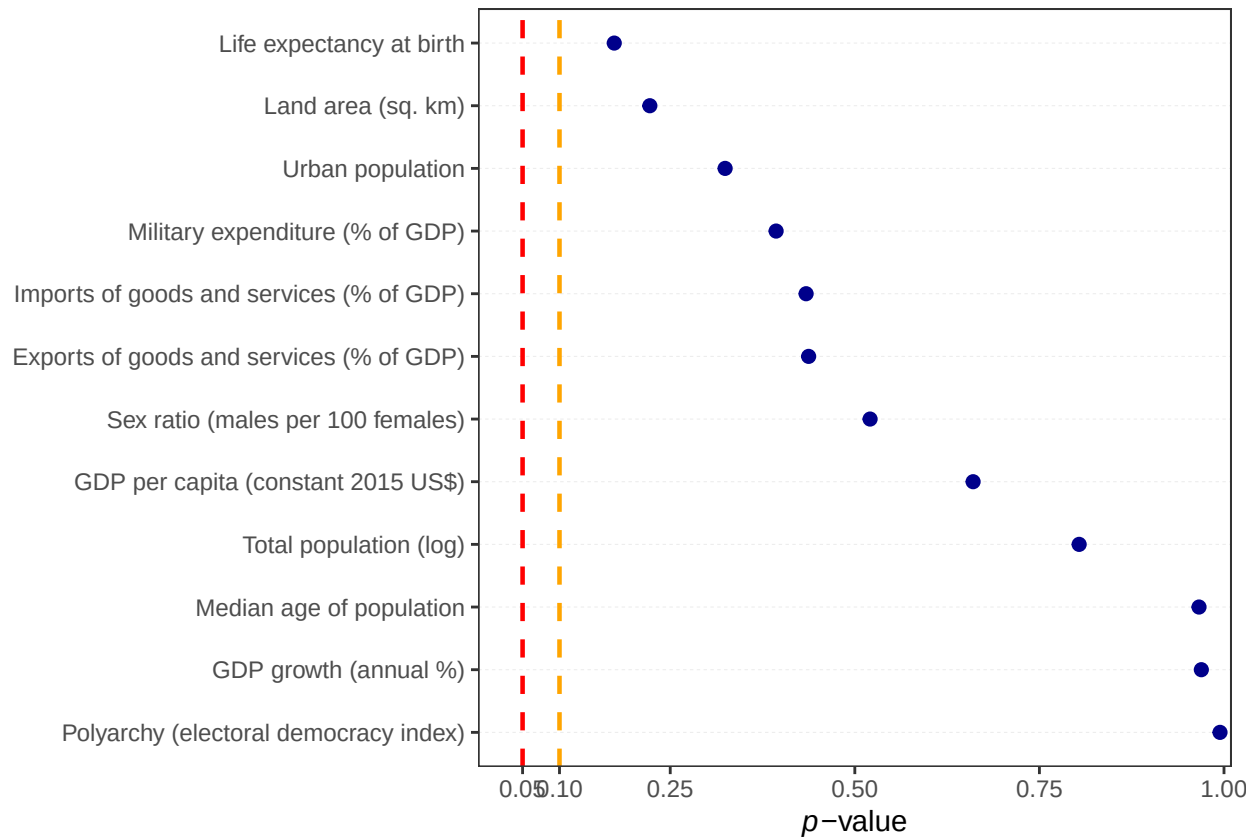
```



```
##
## ===== Dataset: data_bw1 =====
##                                     Variable P_value
## wdi_expmil      Military expenditure (% of GDP) 0.2583
## wdi_pop_log     Total population (log) 0.1372
## wdi_popurb      Urban population 0.0212
## wdi_lifexp      Life expectancy at birth 0.0084
## wdi_export      Exports of goods and services (% of GDP) 0.2502
## wdi_import      Imports of goods and services (% of GDP) 0.4579
## wdi_gdpcapcon2015 GDP per capita (constant 2015 US$) 0.0088
## wdi_gdpgr       GDP growth (annual %) 0.8705
## wdi_area        Land area (sq. km) 0.7441
## wpp_medianage   Median age of population 0.1520
## wpp_sexratio    Sex ratio (males per 100 females) 0.7556
## vdem_polyarchy  Polyarchy (electoral democracy index) 0.1720
```

```
##
## ===== Dataset: data_c0 =====
##                                     Variable P_value
## wdi_expmil      Military expenditure (% of GDP) 0.3934
## wdi_pop_log     Total population (log) 0.8038
## wdi_popurb      Urban population 0.3243
## wdi_lifexp      Life expectancy at birth 0.1742
## wdi_export      Exports of goods and services (% of GDP) 0.4374
## wdi_import      Imports of goods and services (% of GDP) 0.4340
## wdi_gdpcapcon2015 GDP per capita (constant 2015 US$) 0.6602
## wdi_gdpgr       GDP growth (annual %) 0.9693
## wdi_area        Land area (sq. km) 0.2224
## wpp_medianage   Median age of population 0.9662
## wpp_sexratio    Sex ratio (males per 100 females) 0.5207
## vdem_polyarchy  Polyarchy (electoral democracy index) 0.9946
```



b. Covariates Visualization Mean Comparison

```
# datasets to test
datasets <- c("data", "data_bw1", "data_c0") # multiple datasets possible

# Covariates
balance_results <- list()

# plot the substantial differences
for (d in datasets) {
  data_current <- get(d)

  balance_table <- data.frame(
    variable = better_names,
    control_mean = NA,
    treatment_mean = NA
  )

  for (i in seq_along(vars)) {
    v <- vars[i]
    balance_table$treatment_mean[i] <- mean(data_current[[v]][data_current$d == 1], na.rm = TRUE)
    balance_table$control_mean[i] <- mean(data_current[[v]][data_current$d == 0], na.rm = TRUE)
  }

  balance_table <- balance_table %>%
```

```

mutate(
  difference = treatment_mean - control_mean,
  treatment_mean = round(treatment_mean, 2),
  control_mean = round(control_mean, 2),
  difference = round(difference, 2)
)

balance_results[[d]] <- balance_table
}

balance_results[["data"]]

##                                variable control_mean
## wdi_expmil                    Military expenditure (% of GDP)      2.44
## wdi_pop_log                   Total population (log)               16.77
## wdi_popurb                    Urban population                     68.32
## wdi_lifexp                    Life expectancy at birth             71.28
## wdi_export                    Exports of goods and services (% of GDP) 30.26
## wdi_import                    Imports of goods and services (% of GDP) 29.42
## wdi_gdpcapcon2015             GDP per capita (constant 2015 US$)    16207.75
## wdi_gdpgr                     GDP growth (annual %)               2.65
## wdi_area                      Land area (sq. km)                  1854348.06
## wpp_medianage                 Median age of population             29.69
## wpp_sexratio                  Sex ratio (males per 100 females)     99.11
## vdem_polyarchy                Polyarchy (electoral democracy index) 0.64
##                                treatment_mean difference
## wdi_expmil                    2.13      -0.31
## wdi_pop_log                   17.15      0.39
## wdi_popurb                    73.48      5.16
## wdi_lifexp                    74.38      3.10
## wdi_export                    30.45      0.20
## wdi_import                    29.79      0.37
## wdi_gdpcapcon2015             24093.05  7885.30
## wdi_gdpgr                     3.32      0.68
## wdi_area                      1653087.26 -201260.80
## wpp_medianage                 31.47      1.79
## wpp_sexratio                  97.04     -2.07
## vdem_polyarchy                0.72      0.08

balance_results[["data_bw1"]]

##                                variable control_mean
## wdi_expmil                    Military expenditure (% of GDP)      1.96
## wdi_pop_log                   Total population (log)               16.68
## wdi_popurb                    Urban population                     64.74
## wdi_lifexp                    Life expectancy at birth             70.41
## wdi_export                    Exports of goods and services (% of GDP) 27.61
## wdi_import                    Imports of goods and services (% of GDP) 28.52
## wdi_gdpcapcon2015             GDP per capita (constant 2015 US$)    15382.56
## wdi_gdpgr                     GDP growth (annual %)               3.01
## wdi_area                      Land area (sq. km)                  1228860.25
## wpp_medianage                 Median age of population             29.01
## wpp_sexratio                  Sex ratio (males per 100 females)     97.66
## vdem_polyarchy                Polyarchy (electoral democracy index) 0.66

```

```
##          treatment_mean difference
## wdi_expmil           2.34      0.38
## wdi_pop_log          17.07      0.40
## wdi_popurb           71.63      6.88
## wdi_lifexp           74.97      4.56
## wdi_export           31.71      4.10
## wdi_import           30.95      2.43
## wdi_gdpcapcon2015    25102.58   9720.02
## wdi_gdpgr            3.13      0.12
## wdi_area            1406570.13 177709.88
## wpp_medianage        31.69      2.68
## wpp_sexratio         97.35     -0.30
## vdem_polyarchy        0.74      0.08
```

```
balance_results[["data_c0"]]
```

```
##          variable control_mean
## wdi_expmil      Military expenditure (% of GDP)      1.99
## wdi_pop_log      Total population (log)      16.96
## wdi_popurb      Urban population      66.50
## wdi_lifexp      Life expectancy at birth      69.83
## wdi_export      Exports of goods and services (% of GDP)      26.14
## wdi_import      Imports of goods and services (% of GDP)      25.48
## wdi_gdpcapcon2015 GDP per capita (constant 2015 US$)      19633.87
## wdi_gdpgr      GDP growth (annual %)      3.35
## wdi_area      Land area (sq. km)      2291222.48
## wpp_medianage      Median age of population      28.79
## wpp_sexratio      Sex ratio (males per 100 females)      97.04
## vdem_polyarchy      Polyarchy (electoral democracy index)      0.70
##          treatment_mean difference
## wdi_expmil           2.56      0.57
## wdi_pop_log          17.06      0.10
## wdi_popurb           70.92      4.42
## wdi_lifexp           73.19      3.36
## wdi_export           30.22      4.08
## wdi_import           29.14      3.66
## wdi_gdpcapcon2015    22262.16   2628.29
## wdi_gdpgr            3.38      0.03
## wdi_area            1218670.58 -1072551.90
## wpp_medianage        28.90      0.11
## wpp_sexratio         98.12      1.07
## vdem_polyarchy        0.70      0.00
```

c. Manipulation

```
manipulation <- ggplot(data, aes(x = z)) +
  geom_histogram(binwidth = 1,
    color = "black",
    fill = "darkgrey") +
  geom_density(aes(y = ..density.. * nrow(data) * 0.5),
    color = "darkblue",
    linewidth = 1) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "red") +
  scale_y_continuous(
```

```

name = "Frequency",
sec.axis = sec_axis(~ . / (nrow(data) * 0.5),
                     name = "Density")) +
scale_x_continuous(
  breaks = seq(min(data$z, na.rm = TRUE),
               max(data$z, na.rm = TRUE),
               by = 1)) +
labs(x = "Running variable (margin)") +
theme_bw()

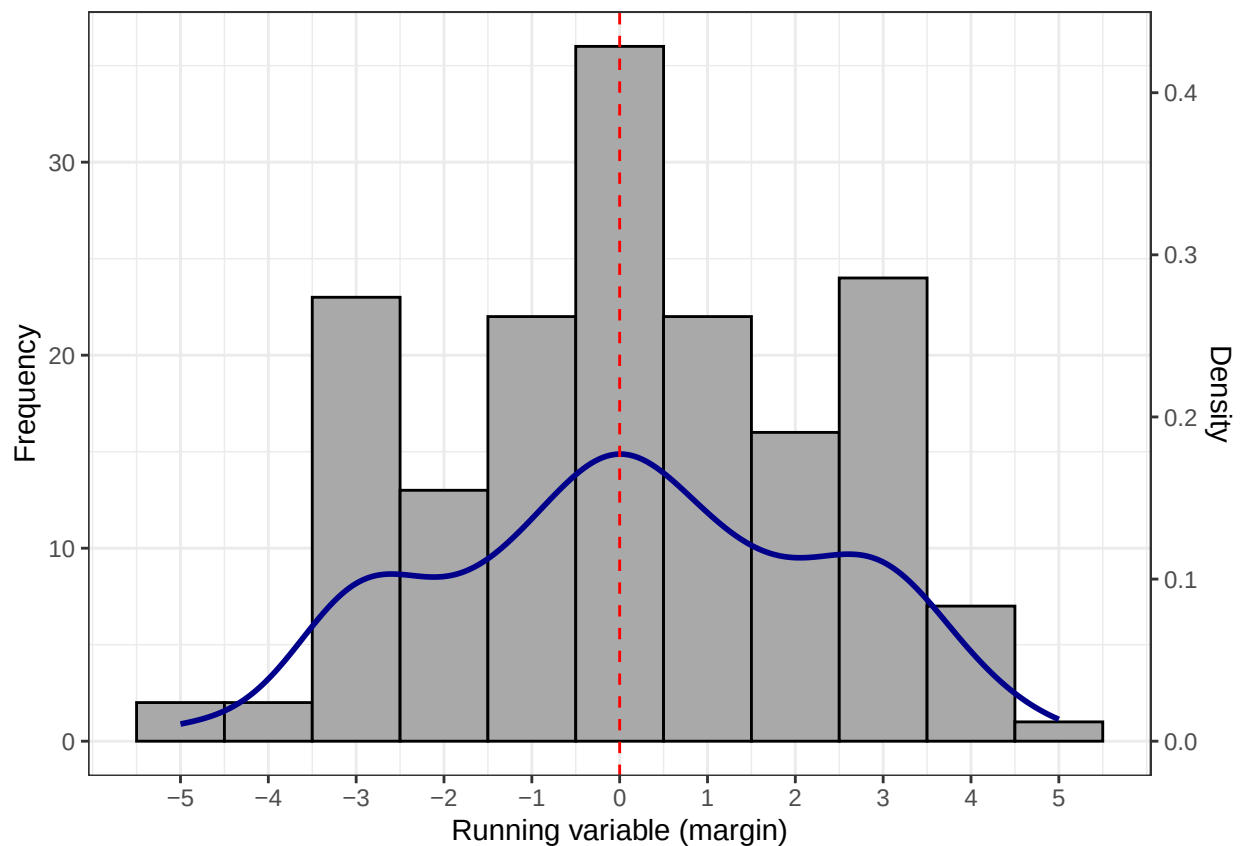
```

manipulation

```

## Warning: The dot-dot notation (`..density..`) was deprecated in ggplot2 3.4.0.
## i Please use `after_stat(density)` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

```



```

ggsave("manipulation.png", manipulation, width = 10, height = 6)

```

```

# McCrary Test-----
#install.packages("rddensity")
library(rddensity)

```

```

## Warning: le package 'rddensity' a été compilé avec la version R 4.5.3

```

```

result <- rddensity(X = data$z, c = 0)
summary(result)

##
## Manipulation testing using local polynomial density estimation.
##
## Number of obs =      168
## Model =            unrestricted
## Kernel =           triangular
## BW method =        estimated
## VCE method =       jackknife
##
## c = 0                Left of c          Right of c
## Number of obs        62                106
## Eff. Number of obs   62                106
## Order est. (p)        2                2
## Order bias (q)        3                3
## BW est. (h)           5                5
##
## Method                T                P > |T|
## Robust                 -1.8578          0.0632

## Warning in summary.CJMrddensity(result): There are repeated observations. Point
## estimates and standard errors have been adjusted. Use option massPoints=FALSE
## to suppress this feature.

##
## P-values of binomial tests (H0: p=0.5).
##
## Window Length / 2      <c      >=c      P>|T|
## 1.000                   22       58      0.0001
## 1.444                   22       58      0.0001
## 1.889                   22       58      0.0001
## 2.333                   35       74      0.0002
## 2.778                   35       74      0.0002
## 3.222                   58       98      0.0017
## 3.667                   58       98      0.0017
## 4.111                   60      105      0.0006
## 4.556                   60      105      0.0006
## 5.000                   62      106      0.0009

```

5. RDD MODELS

a. Run

```

library(rdrobust)

## Warning: le package 'rdrobust' a été compilé avec la version R 4.5.3
# Variable "z" is our running variable
model_h5 <- rdrobust(y = data$y1,
  x = data$z,
  c = 0,
  p = 1,
  kernel = "tri",

```

```

h = 5,
masspoints = "adjust")

summary(model_h5)

## Sharp RD estimates using local polynomial regression.
##
## Number of Obs.                168
## BW type                      Manual
## Kernel                      Triangular
## VCE method                   NN
##
## Number of Obs.                62          106
## Eff. Number of Obs.          60          105
## Order est. (p)                1           1
## Order bias (q)                2           2
## BW est. (h)                   5.000       5.000
## BW bias (b)                   5.000       5.000
## rho (h/b)                     1.000       1.000
## Unique Obs.                   62          106
##
## =====
##              Point      Robust Inference
##              Estimate      z      P>|z|      [ 95% C.I. ]
## -----
##      RD Effect      0.126      0.724      0.469      [-0.275 , 0.597]
## =====

model_mserd <- rdrobust(y = data$y1,
  x = data$z,
  c = 0,
  p = 0,
  kernel = "tri",
  bwselect = "mserd",
  masspoints = "adjust")

## Warning in rdrobust(y = data$y1, x = data$z, c = 0, p = 0, kernel = "tri", :
## Mass points detected in the running variable.

summary(model_mserd)

## Sharp RD estimates using local polynomial regression.
##
## Number of Obs.                168
## BW type                      mserd
## Kernel                      Triangular
## VCE method                   NN
##
## Number of Obs.                62          106
## Eff. Number of Obs.          22          58
## Order est. (p)                0           0
## Order bias (q)                1           1
## BW est. (h)                   1.088       1.088
## BW bias (b)                   2.358       2.358
## rho (h/b)                     0.461       0.461

```

```
## Unique Obs.                5                6
##
## =====
##              Point      Robust Inference
##              Estimate      z      P>|z|      [ 95% C.I. ]
## -----
##      RD Effect      0.072      0.880      0.379      [-0.164 , 0.431]
## =====

model_cerrd <- rdrobust(y = data$y1,
                        x = data$z,
                        c = 0,
                        p = 0,
                        kernel = "tri",
                        bwselect = "cerrd",
                        masspoints = "adjust")

## Warning in rdrobust(y = data$y1, x = data$z, c = 0, p = 0, kernel = "tri", :
## Mass points detected in the running variable.

summary(model_cerrd)

## Sharp RD estimates using local polynomial regression.
##
## Number of Obs.                168
## BW type                      cerrd
## Kernel                      Triangular
## VCE method                    NN
##
## Number of Obs.                62                106
## Eff. Number of Obs.          22                58
## Order est. (p)                0                0
## Order bias (q)                1                1
## BW est. (h)                   1.088            1.088
## BW bias (b)                   2.358            2.358
## rho (h/b)                     0.461            0.461
## Unique Obs.                   5                6
##
## =====
##              Point      Robust Inference
##              Estimate      z      P>|z|      [ 95% C.I. ]
## -----
##      RD Effect      0.072      0.880      0.379      [-0.164 , 0.431]
## =====
```

b. Extract data and print table

```
library(rdrobust)
library(stargazer)

##
## Please cite as:
## Hlavac, Marek (2022). stargazer: Well-Formatted Regression and Summary Statistics Tables.
## R package version 5.2.3. https://CRAN.R-project.org/package=stargazer
```



```

library(webshot)

## Warning: le package 'webshot' a été compilé avec la version R 4.5.3

# model selection
models <- list(
  h5 = model_h5,
  mserd = model_mserd,
  cerrd = model_cerrd
)

# extract the values
extract_rdd_results <- function(model, model_name) {
  data.frame(
    Model = model_name,
    Estimate = model$coef[1],           # Robust estimate
    SE = model$se[3],                   # Robust SE
    CI_lower = model$ci[3,1],           # Robust CI lower
    CI_upper = model$ci[3,2],           # Robust CI upper
    p_value = model$pv[3],              # Robust p-value
    Eff_N_total = sum(model$N_h),       # Total effective N
    Bandwidth = model$bws[1,1],         # Bandwidth
    stringsAsFactors = FALSE
  )
}

results_list <- lapply(names(models), function(name) {
  extract_rdd_results(models[[name]], name)
})

# create the table
results_table <- do.call(rbind, results_list)
results_table_formatted <- results_table %>%
  mutate(
    Estimate = sprintf("%.4f", Estimate),
    CI_95 = sprintf("[%%.4f, %%.4f]", CI_lower, CI_upper),
    `p-value` = sprintf("%.4f", p_value),
    `Eff. N` = sprintf("%d", Eff_N_total),
    `Bandwidth` = sprintf("%.2f", Bandwidth)
  ) %>%
  select(Model, Estimate, CI_95, `p-value`, `Eff. N`, `Bandwidth`)

print(results_table_formatted)

##   Model Estimate          CI_95 p-value Eff. N Bandwidth
## 1   h5   0.1260 [-0.2749, 0.5971]  0.4690   165     5.00
## 2 mserd  0.0717 [-0.1638, 0.4309]  0.3786    80     1.09
## 3 cerrd  0.0717 [-0.1638, 0.4309]  0.3786    80     1.09

# LaTeX visualisation
stargazer(results_table_formatted,
           summary = FALSE,
           rownames = FALSE,
           type = "latex",
           out = "rdd_table.tex")

```

```
##
## % Table created by stargazer v.5.2.3 by Marek Hlavac, Social Policy Institute. E-mail: marek.hlavac@sp.i.cas.cz
## % Date and time: ven., mai 08, 2026 - 17:59:36
## \begin{table}[!htbp] \centering
##   \caption{}
##   \label{}
## \begin{tabular}{@{\extracolsep{5pt}} cccccc}
## \hline
## \hline \hline \hline
## Model & Estimate & CI\_{95} & p-value & Eff. N & Bandwidth \\\
## \hline \hline \hline \hline
## h5 & 0.1260 & [-0.2749, 0.5971] & 0.4690 & 165 & 5.00 \\\
## mserd & 0.0717 & [-0.1638, 0.4309] & 0.3786 & 80 & 1.09 \\\
## cerrd & 0.0717 & [-0.1638, 0.4309] & 0.3786 & 80 & 1.09 \\\
## \hline \hline \hline \hline
## \end{tabular}
## \end{table}
```

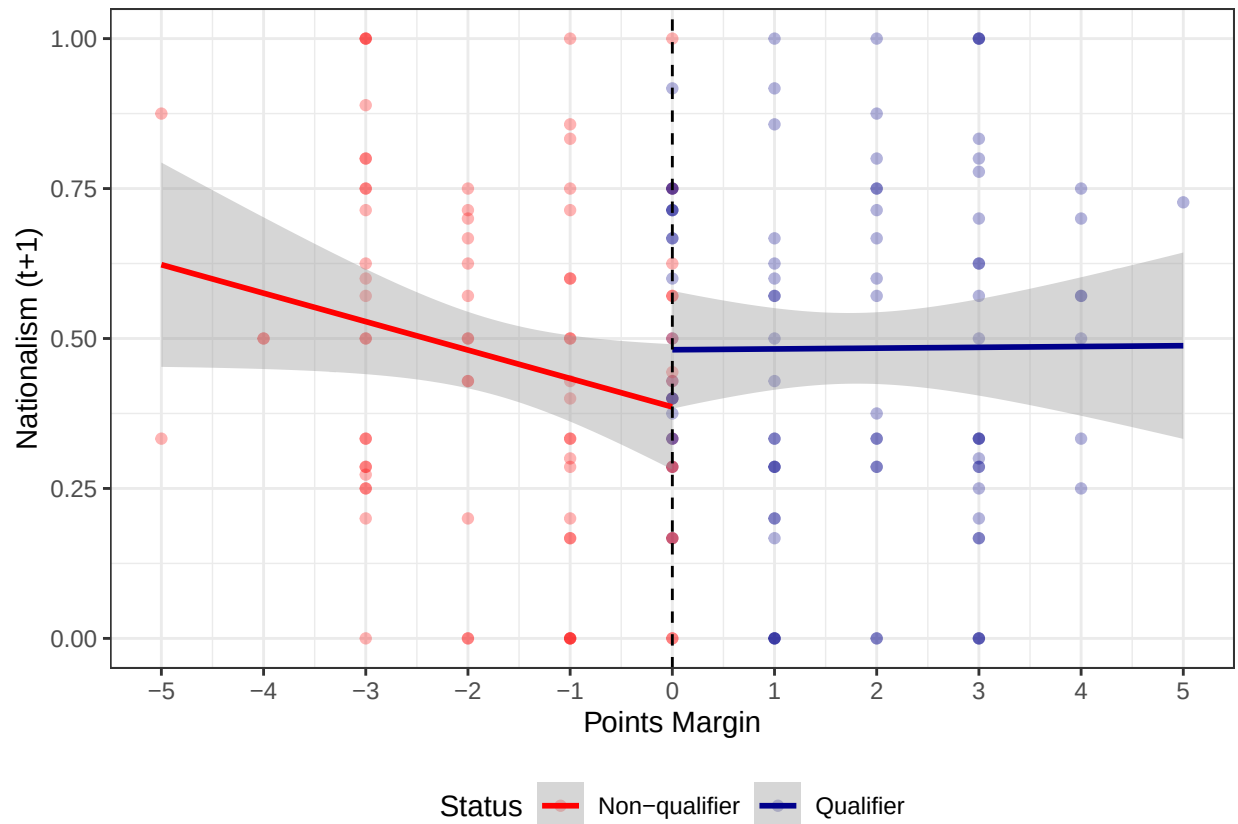
6. RDD VISUALISATION

```
library(ggplot2)
library(dplyr)

# visualisation
p <- ggplot(data, aes(x = z,
                      y = y1,
                      color = factor(d))) +
  geom_point(alpha = 0.3) +
  geom_smooth(method = "lm", se = TRUE) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "black") +
  scale_x_continuous(
    breaks = seq(min(data$z, na.rm = TRUE),
                 max(data$z, na.rm = TRUE), by = 1)
  ) +
  scale_color_manual(
    values = c("0" = "red", "1" = "darkblue"),
    labels = c("Non-qualifier", "Qualifier"),
    name = "Status"
  ) +
  labs(
    x = "Points Margin",
    y = "Nationalism (t+1)"
  ) +
  theme_bw() +
  theme(legend.position = "bottom")

print(p)

## `geom_smooth()` using formula = 'y ~ x'
```



```
ggsave("rdd_plot_y1.png", p, width = 10, height = 6)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

7. ROBUSTNESS CHECK WITHOUT $c = 0$ CASES

a. Refit without the $c = 0$

```
library(rdrobust)
```

```
data_null <- data %>%  
  filter(points_gap != 0)
```

```
nrow(data_null) # 132
```

```
## [1] 132
```

```
attach(data_null)
```

```
## L'objet suivant est masqué _par_ .GlobalEnv:
```

```
##
```

```
##      d
```

```
model_h5_null <- rdrobust(y = y1,
```

```
  x = z,
```

```
  c = 0,
```

```
  p = 1,
```

```

    kernel = "tri",
    h = 5,
    masspoints = "adjust")

summary(model_h5_null)

## Sharp RD estimates using local polynomial regression.
##
## Number of Obs.                132
## BW type                      Manual
## Kernel                      Triangular
## VCE method                   NN
##
## Number of Obs.                62          70
## Eff. Number of Obs.          60          69
## Order est. (p)                1           1
## Order bias (q)                2           2
## BW est. (h)                   5.000       5.000
## BW bias (b)                   5.000       5.000
## rho (h/b)                    1.000       1.000
## Unique Obs.                   62          70
##
## =====
##              Point      Robust Inference
##              Estimate      z      P>|z|      [ 95% C.I. ]
## -----
##      RD Effect      0.042      -0.172      0.863      [-0.628 , 0.526]
## =====

```

b. Extract data and print table

```

library(rdrobust)
library(stargazer)
library(webshot)

models <- list(
  h5 = model_h5_null
)

extract_rdd_results <- function(model, model_name) {
  data.frame(
    Model = model_name,
    Estimate = model$coef[1],           # Estimate
    SE = model$se[3],                  # Robust SE
    CI_lower = model$ci[3,1],          # Robust CI lower
    CI_upper = model$ci[3,2],          # Robust CI upper
    p_value = model$pvalue[3],         # Robust p-value
    Eff_N_total = sum(model$N_h),      # Total effective N
    Bandwidth = model$bws[1,1],        # Bandwidth
    stringsAsFactors = FALSE
  )
}

```

```

results_list <- lapply(names(models), function(name) {
  extract_rdd_results(models[[name]], name)
})

results_table <- do.call(rbind, results_list)
results_table_formatted <- results_table %>%
  mutate(
    Estimate = sprintf("%.4f", Estimate),
    CI_95 = sprintf("[%%.4f, %%.4f]", CI_lower, CI_upper),
    `p-value` = sprintf("%.4f", p_value),
    `Eff. N` = sprintf("%d", Eff_N_total),
    `Bandwidth` = sprintf("%.2f", Bandwidth)
  ) %>%
  select(Model, Estimate, CI_95, `p-value`, `Eff. N`, `Bandwidth`)

print(results_table_formatted)

```

```

##      Model Estimate          CI_95 p-value Eff. N Bandwidth
## 1      h5      0.0423 [-0.6276, 0.5261] 0.8632     129      5.00

```

```

stargazer(results_table_formatted,
           summary = FALSE,
           rownames = FALSE,
           type = "latex",
           out = "rdd_check_table.tex")

```

```

##
## % Table created by stargazer v.5.2.3 by Marek Hlavac, Social Policy Institute. E-mail: marek.hlavac@sp.i.cas.cz
## % Date and time: ven., mai 08, 2026 - 17:59:37
## \begin{table}[!htbp] \centering
##   \caption{}
##   \label{}
##   \begin{tabular}{@{\extracolsep{5pt}} cccccc}
##     \hline
##     \hline \hline \hline
##     Model & Estimate & CI\_95 & p-value & Eff. N & Bandwidth \\
##     \hline \hline \hline \hline
##     h5 & 0.0423 & [-0.6276, 0.5261] & 0.8632 & 129 & 5.00 \\
##     \hline \hline \hline \hline
##   \end{tabular}
## \end{table}

```

c. graphs

```

p_null <- ggplot(data_null, aes(x = z,
                                y = y1,
                                color = factor(d))) +
  geom_point(alpha = 0.3) +
  geom_smooth(method = "lm", se = TRUE) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "black") +
  scale_x_continuous(
    breaks = seq(min(data$z, na.rm = TRUE),
                  max(data$z, na.rm = TRUE), by = 1)
  ) +

```

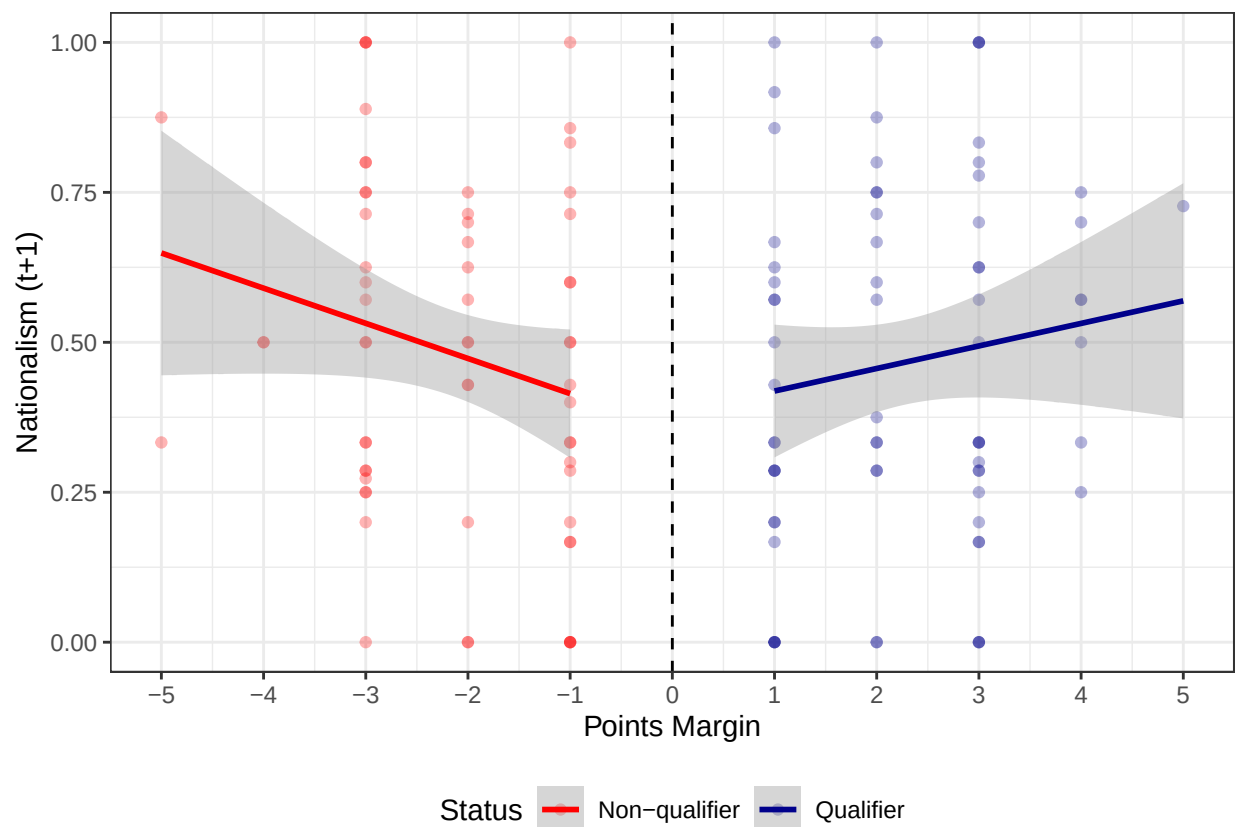
```

scale_color_manual(
  values = c("0" = "red", "1" = "darkblue"),
  labels = c("Non-qualifier", "Qualifier"),
  name = "Status"
) +
labs(
  x = "Points Margin",
  y = "Nationalism (t+1)"
) +
theme_bw() +
theme(legend.position = "bottom")

print(p_null)

```

```
## `geom_smooth()` using formula = 'y ~ x'
```



```
ggsave("rdd_robustness_y1.png", p_null, width = 8, height = 6)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

d. Placebo checks

```

library(rddensity)

rdd_dens <- rddensity(X=data$z,c=0)
summary(rdd_dens)

```

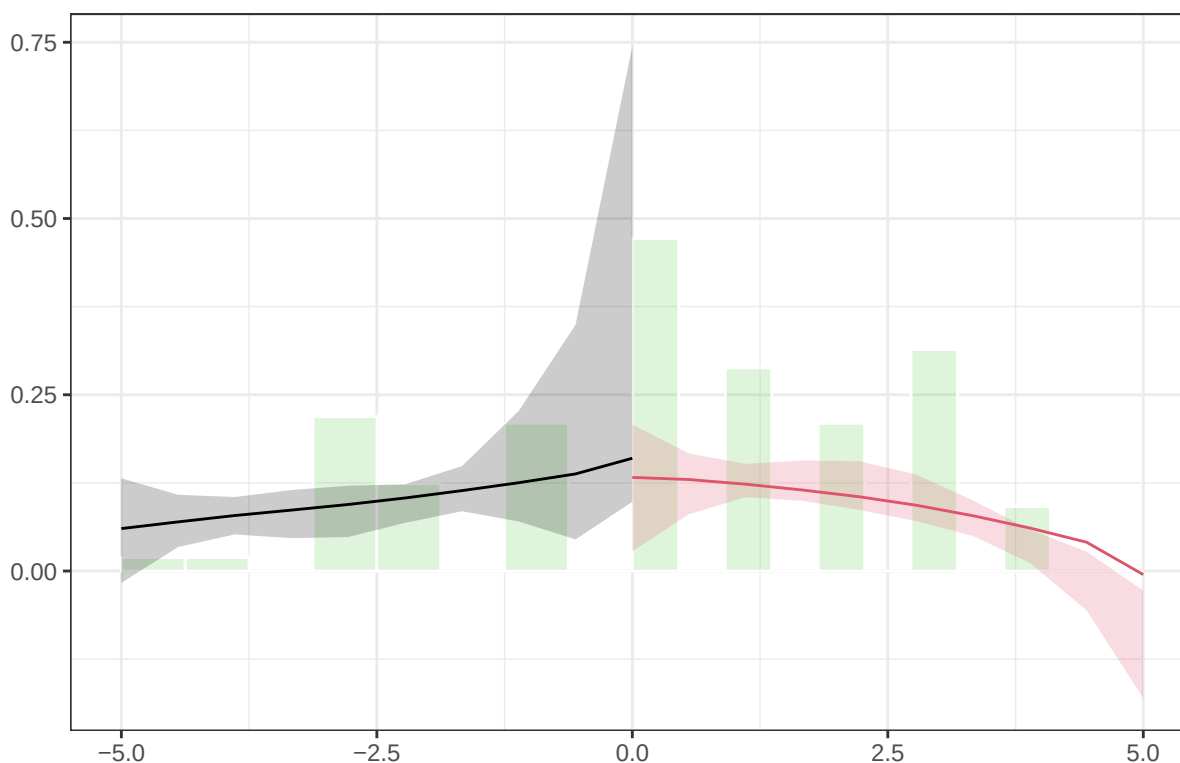
```
##
## Manipulation testing using local polynomial density estimation.
##
## Number of obs =      168
## Model =             unrestricted
## Kernel =            triangular
## BW method =         estimated
## VCE method =        jackknife
##
## c = 0                Left of c          Right of c
## Number of obs        62                106
## Eff. Number of obs   62                106
## Order est. (p)        2                2
## Order bias (q)        3                3
## BW est. (h)          5                5
##
## Method                T                P > |T|
## Robust                -1.8578          0.0632

## Warning in summary.CJMrddensity(rdd_dens): There are repeated observations.
## Point estimates and standard errors have been adjusted. Use option
## massPoints=FALSE to suppress this feature.

##
## P-values of binomial tests (H0: p=0.5).
##
## Window Length / 2      <c      >=c      P>|T|
## 1.000                  22      58      0.0001
## 1.444                  22      58      0.0001
## 1.889                  22      58      0.0001
## 2.333                  35      74      0.0002
## 2.778                  35      74      0.0002
## 3.222                  58      98      0.0017
## 3.667                  58      98      0.0017
## 4.111                  60     105      0.0006
## 4.556                  60     105      0.0006
## 5.000                  62     106      0.0009

rdplotdensity(rdd_dens, data$z)

## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## i The deprecated feature was likely used in the lpdensity package.
## Please report the issue to the authors.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



```
## $Est1
## Call: lpdensity
##
## Sample size                               62
## Polynomial order for point estimation      (p=) 2
## Order of derivative estimated              (v=) 1
## Polynomial order for confidence interval    (q=) 3
## Kernel function                           triangular
## Scaling factor                            0.37125748502994
## Bandwidth method                          user provided
##
## Use summary(...) to show estimates.
##
## $Estr
## Call: lpdensity
##
## Sample size                               106
## Polynomial order for point estimation      (p=) 2
## Order of derivative estimated              (v=) 1
## Polynomial order for confidence interval    (q=) 3
## Kernel function                           triangular
## Scaling factor                            0.634730538922156
## Bandwidth method                          user provided
##
## Use summary(...) to show estimates.
##
```


\$Estplot

